Python LeetCode Top 150

Patterns · Solutions · Templates

Version 2.0 — Deep Coding Patterns Book

What's inside

- Solutions to ~100 must-know LeetCode problems
- Organized by 14 recurring patterns — recognize the pattern, win the problem
- Each solution: problem statement, approach, complexity, code, key insight
- Pattern Selection Guide — pick the right approach in 30 seconds
- Complexity Quick Reference — what's acceptable at each input size
- Code templates for BFS, DFS, binary search, sliding window, DP, backtracking
- Top pitfalls and trick notes interviewers love

EASY	MEDIUM	HARD

Table of Contents

Total: 124 problems · Easy: 36 · Medium: 74 · Hard: 14

1. Arrays & Strings

Bread-and-butter problems. Focus on in-place manipulation, prefix sums, and clever single-pass tricks.

#121 Best Time to Buy and Sell Stock

EASY

PROBLEM

One transaction allowed. Find max profit.

APPROACH

Track min price seen so far. At each step, compute profit if selling today.

COMPLEXITY

O(n) time, O(1) space

PYTHON SOLUTION

```
def maxProfit(prices):
    min_price = float('inf')
    best = 0
    for p in prices:
        min_price = min(min_price, p)
        best = max(best, p - min_price)
    return best
```

#13 Roman to Integer

EASY

PROBLEM

Convert Roman numeral to integer.

APPROACH

If current value < next, subtract; else add. Single pass.

COMPLEXITY

O(n) time, O(1) space

PYTHON SOLUTION

```
def romanToInt(s):
    vals = {'I':1,'V':5,'X':10,'L':50,'C':100,'D':500,'M':1000}
    total = 0
    for i in range(len(s)):
        if i + 1 < len(s) and vals[s[i]] < vals[s[i+1]]:
            total -= vals[s[i]]
        else:
            total += vals[s[i]]
    return total
```

#14 Longest Common Prefix

EASY

PROBLEM

Find longest common prefix among array of strings.

APPROACH

Vertical scan. Take first string; for each char position, check all others.

COMPLEXITY

O(S) where S = total chars

PYTHON SOLUTION

```
def longestCommonPrefix(strs):
    if not strs: return ''
    for i, c in enumerate(strs[0]):
        for s in strs[1:]:
            if i >= len(s) or s[i] != c:
                return strs[0][:i]
    return strs[0]
```

#169 Majority Element

EASY

PROBLEM

Find the element appearing more than n/2 times.

APPROACH

Boyer-Moore voting. Maintain a candidate and count; reset on mismatch.

COMPLEXITY

O(n) time, O(1) space

PYTHON SOLUTION

```
def majorityElement(nums):
    count = 0
    candidate = None
    for n in nums:
        if count == 0:
            candidate = n
        count += 1 if n == candidate else -1
    return candidate
```

■ ***KEY INSIGHT:*** *Boyer-Moore works because the majority element survives all pairwise cancellations.*

#26 Remove Duplicates from Sorted Array

EASY

PROBLEM

Remove duplicates in-place. Return new length k. First k elements should be unique.

APPROACH

Two pointers. Slow tracks unique position; fast scans. Copy when nums[fast] != nums[slow].

COMPLEXITY

O(n) time, O(1) space

PYTHON SOLUTION

```
def removeDuplicates(nums):
    if not nums: return 0
    slow = 0
    for fast in range(1, len(nums)):
        if nums[fast] != nums[slow]:
            slow += 1
            nums[slow] = nums[fast]
    return slow + 1
```

#27 Remove Element

EASY

PROBLEM

Remove all instances of val from nums in-place. Return new length.

APPROACH

Two pointers — slow tracks the write position, fast scans. Copy if not equal to val.

COMPLEXITY

O(n) time, O(1) space

PYTHON SOLUTION

```
def removeElement(nums, val):
    slow = 0
    for fast in range(len(nums)):
        if nums[fast] != val:
            nums[slow] = nums[fast]
            slow += 1
    return slow
```

#28 Find First Occurrence (strStr)

EASY

PROBLEM

Return index of first occurrence of needle in haystack, or -1.

APPROACH

Built-in str.find() in Python. KMP for optimal O(n+m) without built-ins.

COMPLEXITY

O(n*m) naive, O(n+m) KMP

PYTHON SOLUTION

```
def strStr(haystack, needle):
    return haystack.find(needle)

# Manual O(n*m):
def strStr2(h, n):
    if not n: return 0
    for i in range(len(h) - len(n) + 1):
        if h[i:i+len(n)] == n: return i
    return -1
```

#58 Length of Last Word

EASY

PROBLEM

Return length of last word in string s.

APPROACH

Strip trailing spaces; scan from end until space.

COMPLEXITY

O(n) time, O(1) space

PYTHON SOLUTION

```
def lengthOfLastWord(s):
    s = s.rstrip()
    count = 0
    for c in reversed(s):
        if c == ' ': break
        count += 1
    return count
```

#88 Merge Sorted Array

EASY

PROBLEM

Given two sorted arrays nums1 and nums2, merge nums2 into nums1 in-place. nums1 has size m+n with last n slots empty.

APPROACH

Three pointers from the BACK. Avoids shifting elements. i=m-1, j=n-1, k=m+n-1. Compare and place the larger at position k.

COMPLEXITY

O(m+n) time, O(1) space

PYTHON SOLUTION

```
def merge(nums1, m, nums2, n):
    i, j, k = m - 1, n - 1, m + n - 1
    while j >= 0:
        if i >= 0 and nums1[i] > nums2[j]:
            nums1[k] = nums1[i]; i -= 1
        else:
            nums1[k] = nums2[j]; j -= 1
        k -= 1
```

■ ***KEY INSIGHT:*** *From the back avoids overwriting unprocessed elements.*

#12 Integer to Roman

MEDIUM

PROBLEM

Convert integer (1-3999) to Roman.

APPROACH

Greedy with ordered (value, symbol) list including subtractive pairs (CM, CD, XC, XL, IX, IV).

COMPLEXITY

O(1) — bounded number of symbols

PYTHON SOLUTION

```
def intToRoman(num):
    pairs = [(1000,'M'),(900,'CM'),(500,'D'),(400,'CD'),
             (100,'C'),(90,'XC'),(50,'L'),(40,'XL'),
             (10,'X'),(9,'IX'),(5,'V'),(4,'IV'),(1,'I')]
    res = []
    for v, sym in pairs:
        while num >= v:
            res.append(sym); num -= v
    return ''.join(res)
```

#122 Best Time to Buy and Sell Stock II

MEDIUM

PROBLEM

Unlimited transactions allowed.

APPROACH

Greedy: sum every positive difference. Equivalent to buying at every local min, selling at every local max.

COMPLEXITY

O(n) time, O(1) space

PYTHON SOLUTION

```
def maxProfit(prices):
    return sum(max(0, prices[i] - prices[i-1])
               for i in range(1, len(prices)))
```

#134 Gas Station

MEDIUM

PROBLEM

Can you complete a circular route? Return starting index or -1.

APPROACH

If total gas < total cost, impossible. Else, greedily reset start when running deficit. Single pass.

COMPLEXITY

O(n) time, O(1) space

PYTHON SOLUTION

```
def canCompleteCircuit(gas, cost):
    if sum(gas) < sum(cost): return -1
    tank = start = 0
    for i in range(len(gas)):
        tank += gas[i] - cost[i]
        if tank < 0:
            start = i + 1
            tank = 0
    return start
```

#151 Reverse Words in a String

MEDIUM

PROBLEM

Reverse the order of words. Trim and collapse spaces.

APPROACH

split() with no args handles all whitespace, then reverse and join.

COMPLEXITY

O(n) time, O(n) space

PYTHON SOLUTION

```
def reverseWords(s):
    return ' '.join(reversed(s.split()))
```

#189 Rotate Array

MEDIUM

PROBLEM

Rotate array right by k steps.

APPROACH

Reverse trick: reverse all, then reverse first k, then reverse rest. Three O(n) reversals = O(n) total, O(1) space.

COMPLEXITY

O(n) time, O(1) space

PYTHON SOLUTION

```
def rotate(nums, k):
    n = len(nums)
    k %= n
    def reverse(l, r):
        while l < r:
            nums[l], nums[r] = nums[r], nums[l]
            l += 1; r -= 1
    reverse(0, n - 1)
    reverse(0, k - 1)
    reverse(k, n - 1)
```

■ ***KEY INSIGHT:*** *Reverse trick is a classic interview pattern — memorize it.*

#238 Product of Array Except Self

MEDIUM

PROBLEM

result[i] = product of all nums except nums[i]. No division allowed. O(n) time.

APPROACH

Two passes: left prefix products, then multiply by right suffix products in-place.

COMPLEXITY

O(n) time, O(1) extra space (output array doesn't count)

PYTHON SOLUTION

```
def productExceptSelf(nums):
    n = len(nums)
    res = [1] * n
    # Left prefix
    for i in range(1, n):
        res[i] = res[i-1] * nums[i-1]
    # Right suffix
    right = 1
    for i in range(n - 1, -1, -1):
        res[i] *= right
        right *= nums[i]
    return res
```

■ ***KEY INSIGHT:*** *Prefix/suffix products are a recurring pattern.*

#274 H-Index

MEDIUM

PROBLEM

H-index: h papers with at least h citations each.

APPROACH

Counting sort: bucket[i] = number of papers with i citations (cap at n). Walk from high to low summing; first i where sum >= i is the answer.

COMPLEXITY

O(n) time, O(n) space

PYTHON SOLUTION

```
def hIndex(citations):
    n = len(citations)
    buckets = [0] * (n + 1)
    for c in citations:
        buckets[min(c, n)] += 1
    count = 0
    for i in range(n, -1, -1):
        count += buckets[i]
        if count >= i:
            return i
    return 0
```

#45 Jump Game II

MEDIUM

PROBLEM

Minimum jumps to reach the last index.

APPROACH

BFS-like greedy. Track the end of the current 'level' and the farthest reach within it.

COMPLEXITY

O(n) time, O(1) space

PYTHON SOLUTION

```
def jump(nums):
    jumps = 0
    cur_end = 0
    farthest = 0
    for i in range(len(nums) - 1):
        farthest = max(farthest, i + nums[i])
        if i == cur_end:
            jumps += 1
            cur_end = farthest
    return jumps
```

#55 Jump Game

MEDIUM

PROBLEM

Can you reach the last index? Each element is max jump from that position.

APPROACH

Greedy. Track the farthest reachable index. If we're past the farthest, return False.

COMPLEXITY

O(n) time, O(1) space

PYTHON SOLUTION

```
def canJump(nums):
    farthest = 0
    for i, n in enumerate(nums):
        if i > farthest:
            return False
        farthest = max(farthest, i + n)
    return True
```

#6 Zigzag Conversion

MEDIUM

PROBLEM

Write string in zigzag pattern on numRows rows, then read row by row.

APPROACH

Track current row and direction (down or up). Append chars to per-row lists.

COMPLEXITY

O(n) time, O(n) space

PYTHON SOLUTION

```
def convert(s, numRows):
    if numRows == 1: return s
    rows = [[] for _ in range(numRows)]
    row, step = 0, 1
    for c in s:
        rows[row].append(c)
        if row == 0: step = 1
        elif row == numRows - 1: step = -1
        row += step
    return ''.join(''.join(r) for r in rows)
```

#80 Remove Duplicates II (keep at most 2)

MEDIUM

PROBLEM

Remove duplicates such that each element appears at most twice.

APPROACH

Same two-pointer template, but compare with nums[slow-2] to allow up to 2 copies.

COMPLEXITY

O(n) time, O(1) space

PYTHON SOLUTION

```
def removeDuplicates(nums):
    if len(nums) <= 2: return len(nums)
    slow = 2
    for fast in range(2, len(nums)):
        if nums[fast] != nums[slow - 2]:
            nums[slow] = nums[fast]
            slow += 1
    return slow
```

■ ***KEY INSIGHT:*** *Generalize to k copies: compare with nums[slow-k].*

#42 Trapping Rain Water

HARD

PROBLEM

Given elevation heights, compute trapped water.

APPROACH

Two pointers. Track max_left and max_right; water above the shorter side.

COMPLEXITY

O(n) time, O(1) space

PYTHON SOLUTION

```
def trap(height):
    l, r = 0, len(height) - 1
    ml = mr = water = 0
    while l < r:
        if height[l] < height[r]:
            ml = max(ml, height[l])
            water += ml - height[l]
            l += 1
        else:
            mr = max(mr, height[r])
            water += mr - height[r]
            r -= 1
    return water
```

■ ***KEY INSIGHT:*** *Classic two-pointer hard problem. Key insight: water level = min(max_left, max_right).*

#68 Text Justification

HARD

PROBLEM

Given words and maxWidth, format text to be fully justified.

APPROACH

Greedy line packing. For each line, distribute extra spaces between words (or right-pad last line).

COMPLEXITY

O(n) time

PYTHON SOLUTION

```
def fullJustify(words, maxWidth):
    res = []; i = 0
    while i < len(words):
        # pack greedy
        j, line_len = i, 0
        while j < len(words) and line_len + len(words[j]) + (j - i) <= maxWidth:
            line_len += len(words[j])
            j += 1
        # build line
        spaces = maxWidth - line_len
        gaps = j - i - 1
        if j == len(words) or gaps == 0:
            line = ' '.join(words[i:j]).ljust(maxWidth)
        else:
            base, extra = divmod(spaces, gaps)
            line = ''
            for k in range(i, j):
                line += words[k]
                if k < j - 1:
                    line += ' ' * (base + (1 if k - i < extra else 0))
        res.append(line)
        i = j
    return res
```

2. Two Pointers

Two indices traversing the array, often from both ends. Reduces O(n²) brute force to O(n).

#125 Valid Palindrome

EASY

PROBLEM

Check if string is palindrome considering only alphanumeric chars, ignoring case.

APPROACH

Two pointers from both ends. Skip non-alphanumeric. Compare lowercase.

COMPLEXITY

O(n) time, O(1) space

PYTHON SOLUTION

```
def isPalindrome(s):
    l, r = 0, len(s) - 1
    while l < r:
        while l < r and not s[l].isalnum(): l += 1
        while l < r and not s[r].isalnum(): r -= 1
        if s[l].lower() != s[r].lower(): return False
        l += 1; r -= 1
    return True
```

#392 Is Subsequence

EASY

PROBLEM

Check if s is a subsequence of t.

APPROACH

Two pointers. Advance s pointer on match; always advance t pointer.

COMPLEXITY

O(n) time, O(1) space

PYTHON SOLUTION

```
def isSubsequence(s, t):
    i = j = 0
    while i < len(s) and j < len(t):
        if s[i] == t[j]: i += 1
        j += 1
    return i == len(s)
```

#11 Container With Most Water

MEDIUM

PROBLEM

Find two lines forming a container with the maximum water area.

APPROACH

Two pointers. Area = min(h[l], h[r]) * (r - l). Move the shorter side inward — moving the taller side can't help.

COMPLEXITY

O(n) time, O(1) space

PYTHON SOLUTION

```
def maxArea(height):
    l, r = 0, len(height) - 1
    best = 0
    while l < r:
        area = min(height[l], height[r]) * (r - l)
        best = max(best, area)
        if height[l] < height[r]: l += 1
        else: r -= 1
    return best
```

■ ***KEY INSIGHT:*** *Why move the shorter? Moving the taller can only decrease the bounded height while shrinking width.*

#15 3Sum

MEDIUM

PROBLEM

Find all unique triplets that sum to zero.

APPROACH

Sort. For each i, two-pointer on the rest with target = -nums[i]. Skip duplicates carefully.

COMPLEXITY

O(n²) time, O(1) extra

PYTHON SOLUTION

```
def threeSum(nums):
    nums.sort()
    res = []
    for i in range(len(nums) - 2):
        if i > 0 and nums[i] == nums[i-1]: continue
        l, r = i + 1, len(nums) - 1
        while l < r:
            s = nums[i] + nums[l] + nums[r]
            if s == 0:
                res.append([nums[i], nums[l], nums[r]])
                while l < r and nums[l] == nums[l+1]: l += 1
                while l < r and nums[r] == nums[r-1]: r -= 1
                l += 1; r -= 1
            elif s < 0: l += 1
            else: r -= 1
    return res
```

■ ***KEY INSIGHT:*** *Sort + fix one + two-pointer for the rest is the universal kSum template.*

#167 Two Sum II (Sorted)

MEDIUM

PROBLEM

Sorted array. Find two numbers that sum to target. Return 1-indexed positions.

APPROACH

Two pointers from both ends. Move based on sum vs target.

COMPLEXITY

O(n) time, O(1) space

PYTHON SOLUTION

```
def twoSum(nums, target):
    l, r = 0, len(nums) - 1
    while l < r:
        s = nums[l] + nums[r]
        if s == target: return [l + 1, r + 1]
        elif s < target: l += 1
        else: r -= 1
```

3. Sliding Window

Expand window with right pointer; shrink with left when invariant breaks. Classic for substring/subarray problems on contiguous ranges.

#209 Minimum Size Subarray Sum

MEDIUM

PROBLEM

Find min length of contiguous subarray with sum >= target.

APPROACH

Variable window. Expand r adding nums[r]; while sum >= target, shrink l updating answer.

COMPLEXITY

O(n) time, O(1) space

PYTHON SOLUTION

```
def minSubArrayLen(target, nums):
    l = 0; s = 0; best = float('inf')
    for r, x in enumerate(nums):
        s += x
        while s >= target:
            best = min(best, r - l + 1)
            s -= nums[l]; l += 1
    return best if best != float('inf') else 0
```

#3 Longest Substring Without Repeating Characters

MEDIUM

PROBLEM

Find length of longest substring with all unique chars.

APPROACH

Sliding window + dict of char→last index. When duplicate inside window, jump left past the previous occurrence.

COMPLEXITY

O(n) time, O(min(n, charset)) space

PYTHON SOLUTION

```
def lengthOfLongestSubstring(s):
    last = {}
    l = best = 0
    for r, c in enumerate(s):
        if c in last and last[c] >= l:
            l = last[c] + 1
        last[c] = r
        best = max(best, r - l + 1)
    return best
```

#30 Substring with Concatenation of All Words

HARD

PROBLEM

Find all start indices where a substring is concatenation of EACH word in words exactly once.

APPROACH

Sliding window of fixed length = total chars. Use Counter to track word frequencies. Step by word length for each starting offset.

COMPLEXITY

O(n * wordLen)

PYTHON SOLUTION

```
from collections import Counter
def findSubstring(s, words):
    wl = len(words[0])
    n_words = len(words)
    total = wl * n_words
    need = Counter(words)
    res = []
    for offset in range(wl):
        l = offset; have = Counter()
        count = 0
        for r in range(offset, len(s) - wl + 1, wl):
            w = s[r:r+wl]
            if w in need:
                have[w] += 1
                count += 1
                while have[w] > need[w]:
                    have[s[l:l+wl]] -= 1
                    l += wl
                    count -= 1
                if count == n_words:
                    res.append(l)
            else:
                have.clear(); count = 0
                l = r + wl
    return res
```

#76 Minimum Window Substring

HARD

PROBLEM

Find smallest substring of s that contains all chars of t.

APPROACH

Sliding window with two counters (need, have). Track how many UNIQUE chars are satisfied. Shrink when all satisfied.

COMPLEXITY

O(n) time

PYTHON SOLUTION

```
from collections import Counter
def minWindow(s, t):
    if not t: return ''
    need = Counter(t)
    have = {}
    have_count = need_count = len(need)
    l = 0
    best_len = float('inf')
    best_start = 0
    matched = 0
    for r, c in enumerate(s):
        if c in need:
            have[c] = have.get(c, 0) + 1
            if have[c] == need[c]:
                matched += 1
        while matched == need_count:
            if r - l + 1 < best_len:
                best_len = r - l + 1
                best_start = l
            lc = s[l]
            if lc in need:
                have[lc] -= 1
                if have[lc] < need[lc]:
                    matched -= 1
            l += 1
    return '' if best_len == float('inf') else s[best_start:best_start + best_len]
```

■ ***KEY INSIGHT:*** *Track 'matched' chars (unique chars whose count meets need). Cheaper than comparing dicts every iteration.*

4. Hash Map & Sets

O(1) lookup turns many O(n²) brute-force problems into O(n). Watch for hashable keys and collision edge cases.

#1 Two Sum

EASY

PROBLEM

Indices of two numbers summing to target.

APPROACH

Hash map: for each x, check if (target - x) was seen.

COMPLEXITY

O(n) time, O(n) space

PYTHON SOLUTION

```
def twoSum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        if target - x in seen:
            return [seen[target - x], i]
        seen[x] = i
```

#202 Happy Number

EASY

PROBLEM

Repeatedly replace n with sum of squares of digits. Happy = reaches 1; else cycles.

APPROACH

Hash set to detect cycle, OR Floyd's tortoise-and-hare on the digit-sum function.

COMPLEXITY

O(log n) time per step

PYTHON SOLUTION

```
def isHappy(n):
    seen = set()
    while n != 1 and n not in seen:
        seen.add(n)
        n = sum(int(d)**2 for d in str(n))
    return n == 1
```

#205 Isomorphic Strings

EASY

PROBLEM

Two strings are isomorphic if chars in s can be replaced to get t (consistent mapping, no two chars map to same).

APPROACH

Two maps: s→t and t→s. Ensure consistency.

COMPLEXITY

O(n) time

PYTHON SOLUTION

```
def isIsomorphic(s, t):
    if len(s) != len(t): return False
    s2t, t2s = {}, {}
    for a, b in zip(s, t):
        if a in s2t and s2t[a] != b: return False
        if b in t2s and t2s[b] != a: return False
        s2t[a] = b; t2s[b] = a
    return True
```

#219 Contains Duplicate II

EASY

PROBLEM

Does any pair of equal values lie within distance k?

APPROACH

Hash map last_seen[val]. Check distance on collision.

COMPLEXITY

O(n) time

PYTHON SOLUTION

```
def containsNearbyDuplicate(nums, k):
    last = {}
    for i, n in enumerate(nums):
        if n in last and i - last[n] <= k:
            return True
        last[n] = i
    return False
```

#242 Valid Anagram

EASY

PROBLEM

Are s and t anagrams?

APPROACH

Counter comparison or sort comparison.

COMPLEXITY

O(n) time

PYTHON SOLUTION

```
from collections import Counter
def isAnagram(s, t):
    return Counter(s) == Counter(t)
```

#290 Word Pattern

EASY

PROBLEM

Does word sequence follow letter pattern? Like isomorphic but on words.

APPROACH

Same dual-map approach: pattern char ↔ word.

COMPLEXITY

O(n) time

PYTHON SOLUTION

```
def wordPattern(pattern, s):
    words = s.split()
    if len(pattern) != len(words): return False
    p2w, w2p = {}, {}
    for c, w in zip(pattern, words):
        if c in p2w and p2w[c] != w: return False
        if w in w2p and w2p[w] != c: return False
        p2w[c] = w; w2p[w] = c
    return True
```

#383 Ransom Note

EASY

PROBLEM

Can ransomNote be built from magazine letters?

APPROACH

Counter on magazine; subtract for each letter of ransomNote.

COMPLEXITY

O(n) time, O(1) space (alphabet fixed)

PYTHON SOLUTION

```
from collections import Counter
def canConstruct(ransomNote, magazine):
    return not (Counter(ransomNote) - Counter(magazine))
```

#128 Longest Consecutive Sequence

MEDIUM

PROBLEM

Length of longest consecutive elements run. O(n) required.

APPROACH

Put all in a set. For each x, if x-1 not in set, x is a sequence start — count up.

COMPLEXITY

O(n) time, O(n) space

PYTHON SOLUTION

```
def longestConsecutive(nums):
    s = set(nums)
    best = 0
    for n in s:
        if n - 1 not in s:  # only start counting from sequence starts
            cur = n; length = 1
            while cur + 1 in s:
                cur += 1; length += 1
            best = max(best, length)
    return best
```

■ ***KEY INSIGHT:*** *Sequence-start check avoids redundant traversal — keeps it O(n) not O(n²).*

#49 Group Anagrams

MEDIUM

PROBLEM

Group strings that are anagrams.

APPROACH

Hash key by sorted chars (or 26-tuple of counts).

COMPLEXITY

O(n*k log k)

PYTHON SOLUTION

```
from collections import defaultdict
def groupAnagrams(strs):
    groups = defaultdict(list)
    for s in strs:
        groups[''.join(sorted(s))].append(s)
    return list(groups.values())
```

5. Linked List

Pointer manipulation. Master the dummy-node trick, slow/fast pointers, and reversal templates. Draw boxes-and-arrows.

#141 Linked List Cycle

EASY

PROBLEM

Detect if linked list has a cycle.

APPROACH

Floyd's tortoise and hare. If fast meets slow, cycle exists.

COMPLEXITY

O(n) time, O(1) space

PYTHON SOLUTION

```
def hasCycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast: return True
    return False
```

#21 Merge Two Sorted Lists

EASY

PROBLEM

Merge two sorted linked lists.

APPROACH

Dummy node + pointer. Pick smaller head; advance and link.

COMPLEXITY

O(n+m) time, O(1) space

PYTHON SOLUTION

```
def mergeTwoLists(l1, l2):
    dummy = ListNode()
    tail = dummy
    while l1 and l2:
        if l1.val < l2.val:
            tail.next = l1; l1 = l1.next
        else:
            tail.next = l2; l2 = l2.next
        tail = tail.next
    tail.next = l1 or l2
    return dummy.next
```

#138 Copy List with Random Pointer

MEDIUM

PROBLEM

Deep copy linked list where nodes have a random pointer too.

APPROACH

Two-pass with hash map: first pass create clones; second pass set next/random via map. Or O(1) space: interleave clones then split.

COMPLEXITY

O(n) time, O(n) space (map version)

PYTHON SOLUTION

```
def copyRandomList(head):
    if not head: return None
    old_to_new = {}
    cur = head
    while cur:
        old_to_new[cur] = Node(cur.val)
        cur = cur.next
    cur = head
    while cur:
        old_to_new[cur].next = old_to_new.get(cur.next)
        old_to_new[cur].random = old_to_new.get(cur.random)
        cur = cur.next
    return old_to_new[head]
```

#146 LRU Cache

MEDIUM

PROBLEM

Design O(1) get/put cache with LRU eviction.

APPROACH

Doubly linked list + hash map. LRU at tail, MRU at head. Or use OrderedDict.

COMPLEXITY

O(1) per op

PYTHON SOLUTION

```
from collections import OrderedDict
class LRUCache:
    def __init__(self, capacity):
        self.cap = capacity
        self.d = OrderedDict()
    def get(self, key):
        if key not in self.d: return -1
        self.d.move_to_end(key)
        return self.d[key]
    def put(self, key, value):
        if key in self.d: self.d.move_to_end(key)
        self.d[key] = value
        if len(self.d) > self.cap:
            self.d.popitem(last=False)
```

#19 Remove Nth Node From End

MEDIUM

PROBLEM

Remove the nth node from end in one pass.

APPROACH

Two pointers separated by n. When fast hits end, slow is at the node before the target.

COMPLEXITY

O(n) time, O(1) space

PYTHON SOLUTION

```
def removeNthFromEnd(head, n):
    dummy = ListNode(0, head)
    fast = slow = dummy
    for _ in range(n + 1):
        fast = fast.next
    while fast:
        fast = fast.next
        slow = slow.next
    slow.next = slow.next.next
    return dummy.next
```

#61 Rotate List

MEDIUM

PROBLEM

Rotate list to the right by k places.

APPROACH

Find length and tail. Make it circular. Walk to new tail (length - k%length - 1). Cut.

COMPLEXITY

O(n) time

PYTHON SOLUTION

```
def rotateRight(head, k):
    if not head: return None
    length = 1
    tail = head
    while tail.next:
        tail = tail.next; length += 1
    k %= length
    if k == 0: return head
    tail.next = head
    new_tail = head
    for _ in range(length - k - 1):
        new_tail = new_tail.next
    new_head = new_tail.next
    new_tail.next = None
    return new_head
```

#82 Remove Duplicates from Sorted List II

MEDIUM

PROBLEM

Remove ALL nodes that have any duplicate. Keep only distinct values.

APPROACH

Dummy node. If prev.next.val == prev.next.next.val, skip all with that value.

COMPLEXITY

O(n) time

PYTHON SOLUTION

```
def deleteDuplicates(head):
    dummy = ListNode(0, head)
    prev = dummy
    while head:
        if head.next and head.val == head.next.val:
            while head.next and head.val == head.next.val:
                head = head.next
            prev.next = head.next
        else:
            prev = prev.next
        head = head.next
    return dummy.next
```

#86 Partition List

MEDIUM

PROBLEM

Partition list around value x — less than x first, then >= x. Preserve order.

APPROACH

Two dummy heads (less, greater). Stitch them at end.

COMPLEXITY

O(n) time, O(1) space

PYTHON SOLUTION

```
def partition(head, x):
    less = ListNode(); l = less
    greater = ListNode(); g = greater
    while head:
        if head.val < x: l.next = head; l = l.next
        else: g.next = head; g = g.next
        head = head.next
    g.next = None
    l.next = greater.next
    return less.next
```

#92 Reverse Linked List II

MEDIUM

PROBLEM

Reverse nodes from position left to right.

APPROACH

Dummy node. Walk to position before left. Reverse the segment using head-insertion.

COMPLEXITY

O(n) time, O(1) space

PYTHON SOLUTION

```
def reverseBetween(head, left, right):
    dummy = ListNode(0, head)
    prev = dummy
    for _ in range(left - 1):
        prev = prev.next
    cur = prev.next
    # head-insert next nodes after prev
    for _ in range(right - left):
        nxt = cur.next
        cur.next = nxt.next
        nxt.next = prev.next
        prev.next = nxt
    return dummy.next
```

#25 Reverse Nodes in k-Group

HARD

PROBLEM

Reverse nodes in groups of k. Leftover at end stays as-is.

APPROACH

Count k nodes; if available, reverse them; recurse on the rest.

COMPLEXITY

O(n) time, O(1) space iterative

PYTHON SOLUTION

```
def reverseKGroup(head, k):
    dummy = ListNode(0, head)
    group_prev = dummy
    while True:
        kth = group_prev
        for _ in range(k):
            kth = kth.next
            if not kth: return dummy.next
        group_next = kth.next
        # reverse group
        prev, cur = group_next, group_prev.next
        while cur != group_next:
            nxt = cur.next
            cur.next = prev
            prev = cur
            cur = nxt
        tmp = group_prev.next
        group_prev.next = kth
        group_prev = tmp
```

6. Stack

LIFO. Common uses: matching pairs, monotonic stacks for next-greater/smaller problems, expression evaluation.

#20 Valid Parentheses

EASY

PROBLEM

Check if every opening bracket has a matching closing bracket in order.

APPROACH

Stack. Push openers; on closer, pop and match.

COMPLEXITY

O(n) time, O(n) space

PYTHON SOLUTION

```
def isValid(s):
    stack = []
    pairs = {')':'(', ']':'[', '}':'{'}
    for c in s:
        if c in '([{':
            stack.append(c)
        else:
            if not stack or stack.pop() != pairs[c]:
                return False
    return not stack
```

#150 Evaluate Reverse Polish Notation

MEDIUM

PROBLEM

Evaluate postfix expression.

APPROACH

Stack. Push numbers; on operator, pop two and apply.

COMPLEXITY

O(n) time

PYTHON SOLUTION

```
def evalRPN(tokens):
    stack = []
    ops = {'+': lambda a,b: a+b, '-': lambda a,b: a-b,
           '*': lambda a,b: a*b, '/': lambda a,b: int(a/b)}
    for t in tokens:
        if t in ops:
            b = stack.pop(); a = stack.pop()
            stack.append(ops[t](a, b))
        else:
            stack.append(int(t))
    return stack[0]
```

■ ***KEY INSIGHT:*** *int(a/b) handles negative division correctly (Python's // rounds toward -inf).*

#155 Min Stack

MEDIUM

PROBLEM

Stack that returns min in O(1).

APPROACH

Auxiliary stack tracking current min. Or store (val, min_so_far) tuples.

COMPLEXITY

O(1) all ops

PYTHON SOLUTION

```
class MinStack:
    def __init__(self):
        self.stack = []
        self.mins = []
    def push(self, val):
        self.stack.append(val)
        self.mins.append(min(val, self.mins[-1]) if self.mins else val)
    def pop(self):
        self.stack.pop()
        self.mins.pop()
    def top(self):
        return self.stack[-1]
    def getMin(self):
        return self.mins[-1]
```

#71 Simplify Path

MEDIUM

PROBLEM

Simplify Unix path: handle ., .., //, etc.

APPROACH

Split by /. Stack: push real names, pop on '..', ignore '.' and ''.

COMPLEXITY

O(n) time

PYTHON SOLUTION

```
def simplifyPath(path):
    stack = []
    for part in path.split('/'):
        if part == '..':
            if stack: stack.pop()
        elif part and part != '.':
            stack.append(part)
    return '/' + '/'.join(stack)
```

#224 Basic Calculator

HARD

PROBLEM

Evaluate expression with +, -, parentheses.

APPROACH

Stack stores (result, sign) at each '('. Process digits and operators in single pass.

COMPLEXITY

O(n) time

PYTHON SOLUTION

```
def calculate(s):
    stack = []
    result = 0; num = 0; sign = 1
    for c in s:
        if c.isdigit():
            num = num * 10 + int(c)
        elif c in '+-':
            result += sign * num
            num = 0
            sign = 1 if c == '+' else -1
        elif c == '(':
            stack.append(result); stack.append(sign)
            result = 0; sign = 1
        elif c == ')':
            result += sign * num
            num = 0
            result *= stack.pop()   # sign before (
            result += stack.pop()   # result before (
    return result + sign * num
```

7. Trees & BST

Recursion is your friend. Master DFS traversals (preorder, inorder, postorder) and BFS level-order. BSTs: leverage sorted-by-inorder property.

#100 Same Tree

EASY

PROBLEM

Are two trees identical?

APPROACH

Recursive: check root values, then left, then right.

COMPLEXITY

O(n) time

PYTHON SOLUTION

```
def isSameTree(p, q):
    if not p and not q: return True
    if not p or not q: return False
    return (p.val == q.val
            and isSameTree(p.left, q.left)
            and isSameTree(p.right, q.right))
```

#101 Symmetric Tree

EASY

PROBLEM

Is tree a mirror of itself?

APPROACH

Helper compares two subtrees as mirrors: left.left vs right.right, left.right vs right.left.

COMPLEXITY

O(n) time

PYTHON SOLUTION

```
def isSymmetric(root):
    def mirror(a, b):
        if not a and not b: return True
        if not a or not b: return False
        return (a.val == b.val
                and mirror(a.left, b.right)
                and mirror(a.right, b.left))
    return mirror(root.left, root.right) if root else True
```

#104 Maximum Depth of Binary Tree

EASY

PROBLEM

Find the depth.

APPROACH

Recursive: 1 + max(left, right). None → 0.

COMPLEXITY

O(n) time, O(h) space

PYTHON SOLUTION

```
def maxDepth(root):
    if not root: return 0
    return 1 + max(maxDepth(root.left), maxDepth(root.right))
```

#112 Path Sum

EASY

PROBLEM

Does a root-to-leaf path sum to target?

APPROACH

DFS subtracting node value. Return True at leaf if remaining == 0.

COMPLEXITY

O(n) time

PYTHON SOLUTION

```
def hasPathSum(root, target):
    if not root: return False
    if not root.left and not root.right:
        return target == root.val
    return (hasPathSum(root.left, target - root.val)
            or hasPathSum(root.right, target - root.val))
```

#226 Invert Binary Tree

EASY

PROBLEM

Mirror the tree.

APPROACH

Swap left and right at each node; recurse.

COMPLEXITY

O(n) time

PYTHON SOLUTION

```
def invertTree(root):
    if not root: return None
    root.left, root.right = invertTree(root.right), invertTree(root.left)
    return root
```

#530 Minimum Absolute Difference in BST

EASY

PROBLEM

Find min absolute difference between any two node values.

APPROACH

Inorder traversal — adjacent values give min diff in a sorted sequence.

COMPLEXITY

O(n) time

PYTHON SOLUTION

```
def getMinimumDifference(root):
    prev = [None]; best = [float('inf')]
    def inorder(node):
        if not node: return
        inorder(node.left)
        if prev[0] is not None:
            best[0] = min(best[0], node.val - prev[0])
        prev[0] = node.val
        inorder(node.right)
    inorder(root)
    return best[0]
```

#637 Average of Levels

EASY

PROBLEM

Return average value of nodes on each level.

APPROACH

BFS by level, sum and divide.

COMPLEXITY

O(n) time

PYTHON SOLUTION

```
def averageOfLevels(root):
    res = []
    q = deque([root])
    while q:
        n = len(q); s = 0
        for _ in range(n):
            node = q.popleft()
            s += node.val
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
        res.append(s / n)
    return res
```

#102 Binary Tree Level Order Traversal

MEDIUM

PROBLEM

Return values level by level.

APPROACH

BFS with deque, processing one level at a time using current queue size.

COMPLEXITY

O(n) time

PYTHON SOLUTION

```
def levelOrder(root):
    if not root: return []
    res = []
    q = deque([root])
    while q:
        level = []
        for _ in range(len(q)):
            node = q.popleft()
            level.append(node.val)
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
        res.append(level)
    return res
```

#103 Zigzag Level Order Traversal

MEDIUM

PROBLEM

Like 102 but alternate direction each level.

APPROACH

BFS; flip a flag and reverse level on odd levels.

COMPLEXITY

O(n) time

PYTHON SOLUTION

```
def zigzagLevelOrder(root):
    if not root: return []
    res = []; q = deque([root]); ltr = True
    while q:
        level = []
        for _ in range(len(q)):
            node = q.popleft()
            level.append(node.val)
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
        res.append(level if ltr else level[::-1])
        ltr = not ltr
    return res
```

#105 Construct Binary Tree from Preorder and Inorder

MEDIUM

PROBLEM

Build tree from preorder + inorder arrays.

APPROACH

First preorder element is root. Find it in inorder — splits left/right subtrees. Recurse.

COMPLEXITY

O(n) time with index map

PYTHON SOLUTION

```
def buildTree(preorder, inorder):
    idx = {v: i for i, v in enumerate(inorder)}
    self_idx = [0]
    def build(l, r):
        if l > r: return None
        root_val = preorder[self_idx[0]]
        self_idx[0] += 1
        root = TreeNode(root_val)
        mid = idx[root_val]
        root.left = build(l, mid - 1)
        root.right = build(mid + 1, r)
        return root
    return build(0, len(inorder) - 1)
```

#114 Flatten Binary Tree to Linked List

MEDIUM

PROBLEM

Flatten tree into right-pointer linked list in preorder.

APPROACH

Reverse postorder (right, left, root) tracking 'prev'. Set node.right = prev, node.left = None.

COMPLEXITY

O(n) time

PYTHON SOLUTION

```
def flatten(root):
    prev = [None]
    def dfs(node):
        if not node: return
        dfs(node.right)
        dfs(node.left)
        node.right = prev[0]
        node.left = None
        prev[0] = node
    dfs(root)
```

#117 Populating Next Right Pointers II

MEDIUM

PROBLEM

Connect each node's 'next' to its right neighbor in same level. Tree is NOT perfect.

APPROACH

Level-order using already-built 'next' pointers. Dummy node + tail tracks next level.

COMPLEXITY

O(n) time, O(1) extra space

PYTHON SOLUTION

```
def connect(root):
    cur = root
    while cur:
        dummy = Node()
        tail = dummy
        while cur:
            if cur.left:
                tail.next = cur.left
                tail = tail.next
            if cur.right:
                tail.next = cur.right
                tail = tail.next
            cur = cur.next
        cur = dummy.next  # head of next level
    return root
```

#129 Sum Root to Leaf Numbers

MEDIUM

PROBLEM

Each root-to-leaf path forms a number (digits). Sum all such numbers.

APPROACH

DFS carrying current number * 10 + node.val. At leaf, add to total.

COMPLEXITY

O(n) time

PYTHON SOLUTION

```
def sumNumbers(root):
    def dfs(node, cur):
        if not node: return 0
        cur = cur * 10 + node.val
        if not node.left and not node.right:
            return cur
        return dfs(node.left, cur) + dfs(node.right, cur)
    return dfs(root, 0)
```

#199 Binary Tree Right Side View

MEDIUM

PROBLEM

What you see standing on the right side.

APPROACH

Level-order BFS, take last in each level. Or DFS visiting right first and recording first node at each depth.

COMPLEXITY

O(n) time

PYTHON SOLUTION

```
from collections import deque
def rightSideView(root):
    if not root: return []
    res = []
    q = deque([root])
    while q:
        for i in range(len(q)):
            node = q.popleft()
            if i == len(q):  # last in level (after popleft above, len reflects remaining at this level sta
                res.append(node.val)
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
    return res

# Simpler & correct:
def rightSideView2(root):
    if not root: return []
    res = []
    q = deque([root])
    while q:
        size = len(q)
        for i in range(size):
            node = q.popleft()
            if i == size - 1: res.append(node.val)
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
    return res
```

#230 Kth Smallest Element in BST

MEDIUM

PROBLEM

Find kth smallest. Use inorder property of BST.

APPROACH

Inorder traversal (iterative with stack) — yields sorted order. Stop at kth.

COMPLEXITY

O(h + k) time

PYTHON SOLUTION

```
def kthSmallest(root, k):
    stack = []
    while True:
        while root:
            stack.append(root)
            root = root.left
        root = stack.pop()
        k -= 1
        if k == 0: return root.val
        root = root.right
```

#236 Lowest Common Ancestor of Binary Tree

MEDIUM

PROBLEM

Find LCA of two nodes p, q.

APPROACH

Recursive: if root is p or q, return root. Else recurse both sides. If both return non-None, root is LCA. Else return whichever side has a result.

COMPLEXITY

O(n) time

PYTHON SOLUTION

```
def lowestCommonAncestor(root, p, q):
    if not root or root is p or root is q: return root
    left = lowestCommonAncestor(root.left, p, q)
    right = lowestCommonAncestor(root.right, p, q)
    if left and right: return root
    return left or right
```

#98 Validate Binary Search Tree

MEDIUM

PROBLEM

Is it a valid BST?

APPROACH

DFS carrying (lo, hi) bounds. Each node must lie in its bounds.

COMPLEXITY

O(n) time

PYTHON SOLUTION

```
def isValidBST(root):
    def validate(node, lo, hi):
        if not node: return True
        if not (lo < node.val < hi): return False
        return (validate(node.left, lo, node.val)
                and validate(node.right, node.val, hi))
    return validate(root, float('-inf'), float('inf'))
```

■ ***KEY INSIGHT:*** *Naive check (left < root < right at each node) is WRONG — must enforce bounds across descendants.*

#124 Binary Tree Maximum Path Sum

HARD

PROBLEM

Find max path sum. Path goes any-to-any node (can bend at one node).

APPROACH

Post-order DFS. At each node, compute best 'gain' from left/right (clamped to 0). Update global with through-this-node sum. Return best single-side gain.

COMPLEXITY

O(n) time

PYTHON SOLUTION

```
def maxPathSum(root):
    best = [float('-inf')]
    def gain(node):
        if not node: return 0
        left = max(0, gain(node.left))
        right = max(0, gain(node.right))
        best[0] = max(best[0], node.val + left + right)
        return node.val + max(left, right)
    gain(root)
    return best[0]
```

■ ***KEY INSIGHT:*** *Distinguish 'path through node' (counted globally) from 'gain returned upward' (one side only).*

8. Trie

Tree of characters. Best for prefix problems: autocomplete, word search, dictionary lookup.

#208 Implement Trie (Prefix Tree)

MEDIUM

PROBLEM

Implement insert, search, startsWith.

APPROACH

Each node has children dict + end flag.

COMPLEXITY

O(m) per op where m = word length

PYTHON SOLUTION

```
class Trie:
    def __init__(self):
        self.children = {}
        self.end = False
    def insert(self, word):
        node = self
        for c in word:
            if c not in node.children:
                node.children[c] = Trie()
            node = node.children[c]
        node.end = True
    def search(self, word):
        node = self._walk(word)
        return node is not None and node.end
    def startsWith(self, prefix):
        return self._walk(prefix) is not None
    def _walk(self, s):
        node = self
        for c in s:
            if c not in node.children: return None
            node = node.children[c]
        return node
```

#211 Design Add and Search Words (with .)

MEDIUM

PROBLEM

Add words. Search supports '.' wildcard.

APPROACH

Trie + DFS for search. On '.', recurse into all children.

COMPLEXITY

Search worst O(26^m)

PYTHON SOLUTION

```
class WordDictionary:
    def __init__(self):
        self.children = {}
        self.end = False
    def addWord(self, word):
        node = self
        for c in word:
            node.children.setdefault(c, WordDictionary())
            node = node.children[c]
        node.end = True
    def search(self, word):
        def dfs(node, i):
            if i == len(word): return node.end
            c = word[i]
            if c == '.':
                return any(dfs(ch, i + 1) for ch in node.children.values())
            return c in node.children and dfs(node.children[c], i + 1)
        return dfs(self, 0)
```

#212 Word Search II

HARD

PROBLEM

Given a board and word list, find all words on board.

APPROACH

Build Trie of words. DFS from each cell, walking the trie. Prune nodes when found.

COMPLEXITY

O(m*n*4^L) worst, but trie pruning makes it fast

PYTHON SOLUTION

```
def findWords(board, words):
    root = {}
    for w in words:
        node = root
        for c in w:
            node = node.setdefault(c, {})
        node['$'] = w  # mark end
    rows, cols = len(board), len(board[0])
    res = []
    def dfs(r, c, node):
        ch = board[r][c]
        if ch not in node: return
        nxt = node[ch]
        if '$' in nxt:
            res.append(nxt.pop('$'))
        board[r][c] = '#'
        for dr, dc in [(1,0),(-1,0),(0,1),(0,-1)]:
            nr, nc = r+dr, c+dc
            if 0 <= nr < rows and 0 <= nc < cols and board[nr][nc] != '#':
                dfs(nr, nc, nxt)
        board[r][c] = ch
        if not nxt: node.pop(ch)  # prune
    for r in range(rows):
        for c in range(cols):
            dfs(r, c, root)
    return res
```

■ ***KEY INSIGHT:*** *Pruning empty branches and popping found words keeps it fast on large dictionaries.*

9. Graphs (BFS/DFS/Union-Find/Topo)

Master grid DFS/BFS, build adjacency lists, and the topological sort recipes (Kahn's BFS or DFS post-order).

#130 Surrounded Regions

MEDIUM

PROBLEM

Capture O's surrounded by X's. Border-connected O's stay.

APPROACH

DFS from border O's, mark as safe ('#'). Then sweep: '#' → 'O', remaining 'O' → 'X'.

COMPLEXITY

O(m*n) time

PYTHON SOLUTION

```
def solve(board):
    if not board: return
    rows, cols = len(board), len(board[0])
    def dfs(r, c):
        if not (0 <= r < rows and 0 <= c < cols) or board[r][c] != 'O':
            return
        board[r][c] = '#'
        for dr, dc in [(1,0),(-1,0),(0,1),(0,-1)]:
            dfs(r+dr, c+dc)
    for r in range(rows):
        dfs(r, 0); dfs(r, cols - 1)
    for c in range(cols):
        dfs(0, c); dfs(rows - 1, c)
    for r in range(rows):
        for c in range(cols):
            board[r][c] = 'O' if board[r][c] == '#' else 'X'
```

#133 Clone Graph

MEDIUM

PROBLEM

Deep copy a connected undirected graph.

APPROACH

DFS/BFS with hash map from original → clone. Visit each node once.

COMPLEXITY

O(V + E)

PYTHON SOLUTION

```
def cloneGraph(node):
    if not node: return None
    clones = {}
    def dfs(n):
        if n in clones: return clones[n]
        copy = Node(n.val)
        clones[n] = copy
        for nei in n.neighbors:
            copy.neighbors.append(dfs(nei))
        return copy
    return dfs(node)
```

#200 Number of Islands

MEDIUM

PROBLEM

Count connected components of 1s in grid.

APPROACH

DFS or BFS from each unvisited 1, marking visited.

COMPLEXITY

O(m*n) time

PYTHON SOLUTION

```
def numIslands(grid):
    rows, cols = len(grid), len(grid[0])
    count = 0
    def dfs(r, c):
        if not (0 <= r < rows and 0 <= c < cols) or grid[r][c] != '1':
            return
        grid[r][c] = '0'
        for dr, dc in [(1,0),(-1,0),(0,1),(0,-1)]:
            dfs(r+dr, c+dc)
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1':
                count += 1
                dfs(r, c)
    return count
```

#207 Course Schedule (cycle detection)

MEDIUM

PROBLEM

Can you finish all courses given prerequisites? Cycle = no.

APPROACH

Kahn's topo sort: maintain in-degrees, BFS from in-degree-0 nodes. If we process all → no cycle.

COMPLEXITY

O(V + E)

PYTHON SOLUTION

```
from collections import deque, defaultdict
def canFinish(numCourses, prereq):
    graph = defaultdict(list)
    indeg = [0] * numCourses
    for c, p in prereq:
        graph[p].append(c)
        indeg[c] += 1
    q = deque([i for i in range(numCourses) if indeg[i] == 0])
    done = 0
    while q:
        u = q.popleft(); done += 1
        for v in graph[u]:
            indeg[v] -= 1
            if indeg[v] == 0: q.append(v)
    return done == numCourses
```

#210 Course Schedule II

MEDIUM

PROBLEM

Return a valid order to finish courses.

APPROACH

Same Kahn's algorithm — record nodes as they're dequeued.

COMPLEXITY

O(V + E)

PYTHON SOLUTION

```
def findOrder(numCourses, prereq):
    graph = defaultdict(list)
    indeg = [0] * numCourses
    for c, p in prereq:
        graph[p].append(c); indeg[c] += 1
    q = deque([i for i in range(numCourses) if indeg[i] == 0])
    order = []
    while q:
        u = q.popleft(); order.append(u)
        for v in graph[u]:
            indeg[v] -= 1
            if indeg[v] == 0: q.append(v)
    return order if len(order) == numCourses else []
```

#684 Redundant Connection (Union-Find)

MEDIUM

PROBLEM

Tree was 1 edge added to form cycle. Find that edge.

APPROACH

Union-Find. For each edge (u, v), if find(u)==find(v), this is the cycle-causing edge.

COMPLEXITY

O(N α(N))

PYTHON SOLUTION

```
def findRedundantConnection(edges):
    parent = list(range(len(edges) + 1))
    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x
    for u, v in edges:
        pu, pv = find(u), find(v)
        if pu == pv: return [u, v]
        parent[pu] = pv
```

#994 Rotting Oranges (Multi-source BFS)

MEDIUM

PROBLEM

Each minute, rotten oranges (2) infect adjacent fresh (1). Find min minutes till all rotten, or -1.

APPROACH

BFS from all initial rotten cells simultaneously. Track time as level.

COMPLEXITY

O(m*n) time

PYTHON SOLUTION

```
def orangesRotting(grid):
    rows, cols = len(grid), len(grid[0])
    q = deque()
    fresh = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 2: q.append((r, c, 0))
            elif grid[r][c] == 1: fresh += 1
    time = 0
    while q:
        r, c, t = q.popleft()
        time = max(time, t)
        for dr, dc in [(1,0),(-1,0),(0,1),(0,-1)]:
            nr, nc = r+dr, c+dc
            if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 1:
                grid[nr][nc] = 2
                fresh -= 1
                q.append((nr, nc, t + 1))
    return time if fresh == 0 else -1
```

#127 Word Ladder

HARD

PROBLEM

Shortest transformation sequence from begin to end. One letter change per step. Each intermediate in wordList.

APPROACH

BFS. For each word, try changing each letter to each of 26. Or pattern map: '*ot' → ['hot', 'dot'].

COMPLEXITY

O(L * 26 * N) where L = word length

PYTHON SOLUTION

```
from collections import defaultdict, deque
def ladderLength(beginWord, endWord, wordList):
    if endWord not in wordList: return 0
    L = len(beginWord)
    patterns = defaultdict(list)
    for w in wordList:
        for i in range(L):
            patterns[w[:i] + '*' + w[i+1:]].append(w)
    visited = {beginWord}
    q = deque([(beginWord, 1)])
    while q:
        word, steps = q.popleft()
        if word == endWord: return steps
        for i in range(L):
            pat = word[:i] + '*' + word[i+1:]
            for nei in patterns[pat]:
                if nei not in visited:
                    visited.add(nei)
                    q.append((nei, steps + 1))
            patterns[pat] = []  # prune
    return 0
```

■ ***KEY INSIGHT:*** *Pattern map avoids quadratic neighbor checks.*

10. Heap / Priority Queue

heapq is a min-heap on a list. For max-heap, negate values. Used for top-k, scheduling, k-way merge.

#215 Kth Largest Element

MEDIUM

PROBLEM

Find kth largest in unsorted array.

APPROACH

Min-heap of size k. Push and pop if size > k. Top is kth largest. Or heapq.nlargest.

COMPLEXITY

O(n log k) heap, O(n) avg quickselect

PYTHON SOLUTION

```
import heapq
def findKthLargest(nums, k):
    h = []
    for n in nums:
        heapq.heappush(h, n)
        if len(h) > k: heapq.heappop(h)
    return h[0]

# Or simply:
def findKthLargest2(nums, k):
    return heapq.nlargest(k, nums)[-1]
```

#373 Find K Pairs with Smallest Sums

MEDIUM

PROBLEM

Two sorted arrays. Find k pairs (u, v) with smallest sums.

APPROACH

Min-heap. Start with (nums1[0] + nums2[j]) for all j. Or push (sum, i, j) and expand neighbors lazily.

COMPLEXITY

O(k log k)

PYTHON SOLUTION

```
def kSmallestPairs(nums1, nums2, k):
    if not nums1 or not nums2: return []
    heap = [(nums1[0] + nums2[j], 0, j) for j in range(min(k, len(nums2)))]
    heapq.heapify(heap)
    res = []
    while heap and len(res) < k:
        s, i, j = heapq.heappop(heap)
        res.append([nums1[i], nums2[j]])
        if i + 1 < len(nums1):
            heapq.heappush(heap, (nums1[i+1] + nums2[j], i + 1, j))
    return res
```

#295 Find Median from Data Stream

HARD

PROBLEM

Add numbers, query median anytime.

APPROACH

Two heaps: max-heap (lower half) + min-heap (upper half). Balance sizes. Median = top of larger, or avg of tops.

COMPLEXITY

O(log n) add, O(1) findMedian

PYTHON SOLUTION

```
class MedianFinder:
    def __init__(self):
        self.lower = []  # max-heap via negation
        self.upper = []  # min-heap
    def addNum(self, num):
        heapq.heappush(self.lower, -num)
        heapq.heappush(self.upper, -heapq.heappop(self.lower))
        if len(self.upper) > len(self.lower):
            heapq.heappush(self.lower, -heapq.heappop(self.upper))
    def findMedian(self):
        if len(self.lower) > len(self.upper):
            return -self.lower[0]
        return (-self.lower[0] + self.upper[0]) / 2
```

■ ***KEY INSIGHT:*** *Two-heap pattern: invariant is len(lower) == len(upper) or len(lower) == len(upper) + 1.*

#502 IPO (max profit with k projects, capital limit)

HARD

PROBLEM

Pick at most k projects to maximize capital. Each project has cost (need current capital >= cost) and profit.

APPROACH

Two heaps: min-heap on cost (to find affordable), max-heap on profit (pick best affordable).

COMPLEXITY

O((n + k) log n)

PYTHON SOLUTION

```
def findMaximizedCapital(k, w, profits, capital):
    projects = sorted(zip(capital, profits))
    available = []
    i = 0
    for _ in range(k):
        while i < len(projects) and projects[i][0] <= w:
            heapq.heappush(available, -projects[i][1])
            i += 1
        if not available: break
        w += -heapq.heappop(available)
    return w
```

11. Backtracking

Recipe: choose → recurse → un-choose. Build solutions incrementally and abandon a branch as soon as it can't lead to a valid solution.

#17 Letter Combinations of Phone Number

MEDIUM

PROBLEM

All letter combos for a digit string (phone keypad).

APPROACH

Recursion: for each digit, append each possible letter.

COMPLEXITY

O(4^n)

PYTHON SOLUTION

```
def letterCombinations(digits):
    if not digits: return []
    keys = {'2':'abc','3':'def','4':'ghi','5':'jkl',
            '6':'mno','7':'pqrs','8':'tuv','9':'wxyz'}
    res = []
    def bt(i, cur):
        if i == len(digits):
            res.append(cur); return
        for c in keys[digits[i]]:
            bt(i + 1, cur + c)
    bt(0, '')
    return res
```

#22 Generate Parentheses

MEDIUM

PROBLEM

All valid parenthesis combos of n pairs.

APPROACH

Backtrack tracking (open_used, close_used). Open if open < n; close if close < open.

COMPLEXITY

Catalan number

PYTHON SOLUTION

```
def generateParenthesis(n):
    res = []
    def bt(s, opn, cls):
        if len(s) == 2 * n:
            res.append(s); return
        if opn < n: bt(s + '(', opn + 1, cls)
        if cls < opn: bt(s + ')', opn, cls + 1)
    bt('', 0, 0)
    return res
```

#39 Combination Sum

MEDIUM

PROBLEM

Find combinations of candidates summing to target. Each may be reused.

APPROACH

Backtrack. Pass current index to avoid duplicates; pass same i to allow reuse.

COMPLEXITY

Exponential

PYTHON SOLUTION

```
def combinationSum(candidates, target):
    res = []
    candidates.sort()
    def bt(start, path, remain):
        if remain == 0:
            res.append(path[:])
            return
        for i in range(start, len(candidates)):
            if candidates[i] > remain: break
            path.append(candidates[i])
            bt(i, path, remain - candidates[i])
            path.pop()
    bt(0, [], target)
    return res
```

#46 Permutations

MEDIUM

PROBLEM

All permutations of distinct integers.

APPROACH

Swap-in-place backtrack, or use a 'used' boolean array.

COMPLEXITY

O(n * n!)

PYTHON SOLUTION

```
def permute(nums):
    res = []
    def bt(start):
        if start == len(nums):
            res.append(nums[:])
            return
        for i in range(start, len(nums)):
            nums[start], nums[i] = nums[i], nums[start]
            bt(start + 1)
            nums[start], nums[i] = nums[i], nums[start]
    bt(0)
    return res
```

#77 Combinations

MEDIUM

PROBLEM

All combinations of k numbers from 1..n.

APPROACH

Backtrack: at each step, choose i and recurse on i+1..n.

COMPLEXITY

O(C(n,k) * k)

PYTHON SOLUTION

```
def combine(n, k):
    res = []
    def bt(start, path):
        if len(path) == k:
            res.append(path[:])
            return
        for i in range(start, n + 1):
            path.append(i)
            bt(i + 1, path)
            path.pop()
    bt(1, [])
    return res
```

#78 Subsets

MEDIUM

PROBLEM

All subsets of distinct integers.

APPROACH

Backtrack: at each index, take or skip. Or iterative: start with [[]], for each num extend with num appended.

COMPLEXITY

O(n * 2^n)

PYTHON SOLUTION

```
def subsets(nums):
    res = []
    def bt(i, path):
        if i == len(nums):
            res.append(path[:])
            return
        # skip
        bt(i + 1, path)
        # take
        path.append(nums[i])
        bt(i + 1, path)
        path.pop()
    bt(0, [])
    return res
```

#79 Word Search

MEDIUM

PROBLEM

Does word exist on grid? Adjacent cells, no reuse.

APPROACH

DFS from each cell; mark visited with sentinel; backtrack.

COMPLEXITY

O(m*n*4^L)

PYTHON SOLUTION

```
def exist(board, word):
    rows, cols = len(board), len(board[0])
    def dfs(r, c, i):
        if i == len(word): return True
        if not (0 <= r < rows and 0 <= c < cols): return False
        if board[r][c] != word[i]: return False
        board[r][c] = '#'  # mark
        ok = (dfs(r+1, c, i+1) or dfs(r-1, c, i+1)
              or dfs(r, c+1, i+1) or dfs(r, c-1, i+1))
        board[r][c] = word[i]  # restore
        return ok
    for r in range(rows):
        for c in range(cols):
            if dfs(r, c, 0): return True
    return False
```

#51 N-Queens

HARD

PROBLEM

Place n queens so no two attack each other.

APPROACH

Backtrack by row. Track used columns, diagonals (r+c), anti-diagonals (r-c).

COMPLEXITY

O(n!)

PYTHON SOLUTION

```
def solveNQueens(n):
    res = []
    cols, diag, anti = set(), set(), set()
    board = [['.'] * n for _ in range(n)]
    def bt(r):
        if r == n:
            res.append([''.join(row) for row in board])
            return
        for c in range(n):
            if c in cols or (r+c) in diag or (r-c) in anti: continue
            cols.add(c); diag.add(r+c); anti.add(r-c)
            board[r][c] = 'Q'
            bt(r + 1)
            board[r][c] = '.'
            cols.remove(c); diag.remove(r+c); anti.remove(r-c)
    bt(0)
    return res
```

12. Dynamic Programming

Recipe: 1) Define state 2) Recurrence 3) Base case 4) Iteration order 5) Answer location 6) Space optimize. Top-down (memoization) or bottom-up (tabulation).

#70 Climbing Stairs

EASY

PROBLEM

n stairs, 1 or 2 steps at a time. How many ways?

APPROACH

Fibonacci: ways(n) = ways(n-1) + ways(n-2).

COMPLEXITY

O(n) time, O(1) space

PYTHON SOLUTION

```
def climbStairs(n):
    if n <= 2: return n
    a, b = 1, 2
    for _ in range(3, n + 1):
        a, b = b, a + b
    return b
```

#139 Word Break

MEDIUM

PROBLEM

Can s be segmented into dictionary words?

APPROACH

dp[i] = True iff s[:i] is breakable. dp[i] = any(dp[j] and s[j:i] in words for j < i).

COMPLEXITY

$O(n^2)$

PYTHON SOLUTION

```
def wordBreak(s, words):
    words = set(words)
    n = len(s)
    dp = [False] * (n + 1)
    dp[0] = True
    for i in range(1, n + 1):
        for j in range(i):
            if dp[j] and s[j:i] in words:
                dp[i] = True
                break
    return dp[n]
```

#152 Maximum Product Subarray

MEDIUM

PROBLEM

Find contiguous subarray with max product.

APPROACH

Track both max AND min ending here (negative * negative = positive can flip).

COMPLEXITY

O(n)

PYTHON SOLUTION

```
def maxProduct(nums):
    cur_max = cur_min = res = nums[0]
    for x in nums[1:]:
        if x < 0: cur_max, cur_min = cur_min, cur_max
        cur_max = max(x, x * cur_max)
        cur_min = min(x, x * cur_min)
        res = max(res, cur_max)
    return res
```

■ ***KEY INSIGHT:*** *Swap max/min on negative — the smallest becomes the largest after multiplication.*

#198 House Robber

MEDIUM

PROBLEM

Can't rob adjacent houses. Max loot?

APPROACH

dp[i] = max(dp[i-1], dp[i-2] + nums[i]).

COMPLEXITY

O(n) time, O(1) space

PYTHON SOLUTION

```
def rob(nums):
    prev = curr = 0
    for n in nums:
        prev, curr = curr, max(curr, prev + n)
    return curr
```

#221 Maximal Square

MEDIUM

PROBLEM

Largest square of 1s in a binary matrix.

APPROACH

dp[i][j] = side length of largest square with bottom-right at (i,j) = 1 + min(top, left, top-left) if matrix[i][j]=='1'.

COMPLEXITY

O(m*n)

PYTHON SOLUTION

```
def maximalSquare(matrix):
    if not matrix: return 0
    rows, cols = len(matrix), len(matrix[0])
    dp = [[0]*(cols+1) for _ in range(rows+1)]
    best = 0
    for i in range(1, rows+1):
        for j in range(1, cols+1):
            if matrix[i-1][j-1] == '1':
                dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])
                best = max(best, dp[i][j])
    return best * best
```

#300 Longest Increasing Subsequence

MEDIUM

PROBLEM

Length of longest strictly increasing subsequence.

APPROACH

Two solutions: (1) O(n²) DP: dp[i] = 1 + max(dp[j] for j<i, nums[j]<nums[i]). (2) O(n log n) patience sort: maintain 'tails', binary search.

COMPLEXITY

O(n log n) optimal

PYTHON SOLUTION

```
from bisect import bisect_left
def lengthOfLIS(nums):
    tails = []
    for x in nums:
        i = bisect_left(tails, x)
        if i == len(tails): tails.append(x)
        else: tails[i] = x
    return len(tails)
```

■ ***KEY INSIGHT:*** *tails[i] = smallest tail of an LIS of length i+1. The array isn't an actual LIS but length is correct.*

#322 Coin Change

MEDIUM

PROBLEM

Min coins to make amount.

APPROACH

dp[a] = min(dp[a-c] + 1) for each c.

COMPLEXITY

O(amount * #coins)

PYTHON SOLUTION

```
def coinChange(coins, amount):
    INF = float('inf')
    dp = [INF] * (amount + 1)
    dp[0] = 0
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a:
                dp[a] = min(dp[a], dp[a-c] + 1)
    return dp[amount] if dp[amount] != INF else -1
```

#5 Longest Palindromic Substring

MEDIUM

PROBLEM

Find longest palindromic substring.

APPROACH

Expand around each center (handles odd and even lengths).

COMPLEXITY

O(n²) time, O(1) space

PYTHON SOLUTION

```
def longestPalindrome(s):
    if not s: return ''
    def expand(l, r):
        while l >= 0 and r < len(s) and s[l] == s[r]:
            l -= 1; r += 1
        return s[l+1:r]
    best = ''
    for i in range(len(s)):
        for cand in (expand(i, i), expand(i, i+1)):
            if len(cand) > len(best): best = cand
    return best
```

#53 Maximum Subarray (Kadane's)

MEDIUM

PROBLEM

Find contiguous subarray with max sum.

APPROACH

Kadane: cur = max(num, cur + num); res = max(res, cur).

COMPLEXITY

O(n)

PYTHON SOLUTION

```
def maxSubArray(nums):
    cur = res = nums[0]
    for x in nums[1:]:
        cur = max(x, cur + x)
        res = max(res, cur)
    return res
```

■ ***KEY INSIGHT:*** *Kadane is the foundation for many max-subarray variants. Memorize it.*

#62 Unique Paths

MEDIUM

PROBLEM

Count paths in m×n grid moving only right/down.

APPROACH

dp[i][j] = dp[i-1][j] + dp[i][j-1]. Space-optimized to one row.

COMPLEXITY

O(m*n) time, O(n) space

PYTHON SOLUTION

```
def uniquePaths(m, n):
    dp = [1] * n
    for _ in range(1, m):
        for j in range(1, n):
            dp[j] += dp[j-1]
    return dp[-1]
```

#64 Minimum Path Sum

MEDIUM

PROBLEM

Min sum of path from top-left to bottom-right.

APPROACH

dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1]). In-place possible.

COMPLEXITY

O(m*n)

PYTHON SOLUTION

```
def minPathSum(grid):
    rows, cols = len(grid), len(grid[0])
    for i in range(rows):
        for j in range(cols):
            if i == 0 and j == 0: continue
            elif i == 0: grid[i][j] += grid[i][j-1]
            elif j == 0: grid[i][j] += grid[i-1][j]
            else: grid[i][j] += min(grid[i-1][j], grid[i][j-1])
    return grid[-1][-1]
```

#72 Edit Distance (Levenshtein)

HARD

PROBLEM

Min ops (insert, delete, replace) to transform s1 to s2.

APPROACH

dp[i][j] = edit distance for s1[:i], s2[:j]. If chars match, dp[i][j] = dp[i-1][j-1]. Else 1 + min of three options.

COMPLEXITY

O(m*n)

PYTHON SOLUTION

```
def minDistance(s1, s2):
    m, n = len(s1), len(s2)
    dp = [[0]*(n+1) for _ in range(m+1)]
    for i in range(m+1): dp[i][0] = i
    for j in range(n+1): dp[0][j] = j
    for i in range(1, m+1):
        for j in range(1, n+1):
            if s1[i-1] == s2[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
                dp[i][j] = 1 + min(dp[i-1][j-1],  # replace
                                   dp[i-1][j],     # delete
                                   dp[i][j-1])     # insert
    return dp[m][n]
```

13. Binary Search

Not just for sorted arrays — anywhere you have a monotonic predicate. Templates for first/last occurrence, leftmost, rightmost.

#35 Search Insert Position

EASY

PROBLEM

Find target index, or where it would go.

APPROACH

bisect_left or hand-rolled.

COMPLEXITY

O(log n)

PYTHON SOLUTION

```
def searchInsert(nums, target):
    lo, hi = 0, len(nums)
    while lo < hi:
        mid = (lo + hi) // 2
        if nums[mid] < target: lo = mid + 1
        else: hi = mid
    return lo
```

#153 Find Min in Rotated Sorted Array

MEDIUM

PROBLEM

Find the minimum of a rotated sorted array.

APPROACH

Binary search compared against nums[hi]. If nums[mid] > nums[hi], min is right; else left (incl. mid).

COMPLEXITY

O(log n)

PYTHON SOLUTION

```
def findMin(nums):
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if nums[mid] > nums[hi]: lo = mid + 1
        else: hi = mid
    return nums[lo]
```

#162 Find Peak Element

MEDIUM

PROBLEM

Find any peak (element greater than neighbors).

APPROACH

Binary search. If nums[mid] < nums[mid+1], peak is on the right; else on the left.

COMPLEXITY

O(log n)

PYTHON SOLUTION

```
def findPeakElement(nums):
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if nums[mid] < nums[mid + 1]: lo = mid + 1
        else: hi = mid
    return lo
```

#33 Search in Rotated Sorted Array

MEDIUM

PROBLEM

Sorted array rotated at unknown pivot. Find target.

APPROACH

Modified binary search. Identify which half is sorted; check if target lies in that half.

COMPLEXITY

O(log n)

PYTHON SOLUTION

```
def search(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if nums[mid] == target: return mid
        if nums[lo] <= nums[mid]:  # left half sorted
            if nums[lo] <= target < nums[mid]: hi = mid - 1
            else: lo = mid + 1
        else:  # right half sorted
            if nums[mid] < target <= nums[hi]: lo = mid + 1
            else: hi = mid - 1
    return -1
```

#74 Search 2D Matrix

MEDIUM

PROBLEM

Matrix where each row is sorted and first of each row > last of prev row.

APPROACH

Treat as flat sorted array; binary search with row,col = divmod.

COMPLEXITY

O(log(m*n))

PYTHON SOLUTION

```
def searchMatrix(matrix, target):
    rows, cols = len(matrix), len(matrix[0])
    lo, hi = 0, rows * cols - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        v = matrix[mid // cols][mid % cols]
        if v == target: return True
        elif v < target: lo = mid + 1
        else: hi = mid - 1
    return False
```

#4 Median of Two Sorted Arrays

HARD

PROBLEM

Find median in O(log(min(m,n))).

APPROACH

Binary search on the smaller array's partition. Partition both so left halves form lower half of merged.

COMPLEXITY

O(log min(m,n))

PYTHON SOLUTION

```
def findMedianSortedArrays(a, b):
    if len(a) > len(b): a, b = b, a
    m, n = len(a), len(b)
    lo, hi = 0, m
    while lo <= hi:
        i = (lo + hi) // 2
        j = (m + n + 1) // 2 - i
        al = a[i-1] if i > 0 else float('-inf')
        ar = a[i] if i < m else float('inf')
        bl = b[j-1] if j > 0 else float('-inf')
        br = b[j] if j < n else float('inf')
        if al <= br and bl <= ar:
            if (m + n) % 2 == 0:
                return (max(al, bl) + min(ar, br)) / 2
            return max(al, bl)
        elif al > br: hi = i - 1
        else: lo = i + 1
```

14. Math & Bit Manipulation

Bit ops: & (and), | (or), ^ (xor), ~ (not), << >> (shifts). XOR cancels duplicates. n & (n-1) clears lowest set bit.

#136 Single Number

EASY

PROBLEM

Every number appears twice except one. Find it.

APPROACH

XOR all — duplicates cancel.

COMPLEXITY

O(n) time, O(1) space

PYTHON SOLUTION

```
def singleNumber(nums):
    x = 0
    for n in nums: x ^= n
    return x
```

#190 Reverse Bits

EASY

PROBLEM

Reverse bits of 32-bit unsigned int.

APPROACH

Pop lowest bit, shift into result.

COMPLEXITY

O(32)

PYTHON SOLUTION

```
def reverseBits(n):
    res = 0
    for _ in range(32):
        res = (res << 1) | (n & 1)
        n >>= 1
    return res
```

#191 Number of 1 Bits

EASY

PROBLEM

Hamming weight (count of 1 bits).

APPROACH

n & (n-1) clears lowest set bit. Count iterations.

COMPLEXITY

O(set bits)

PYTHON SOLUTION

```
def hammingWeight(n):
    count = 0
    while n:
        n &= n - 1
        count += 1
    return count

# Or: bin(n).count('1')
```

#67 Add Binary

EASY

PROBLEM

Sum of two binary strings.

APPROACH

Right-to-left with carry. Or int(a,2)+int(b,2) → bin().

COMPLEXITY

O(max(m,n))

PYTHON SOLUTION

```
def addBinary(a, b):
    i, j = len(a) - 1, len(b) - 1
    carry = 0; res = []
    while i >= 0 or j >= 0 or carry:
        s = carry
        if i >= 0: s += int(a[i]); i -= 1
        if j >= 0: s += int(b[j]); j -= 1
        carry = s // 2
        res.append(str(s % 2))
    return ''.join(reversed(res))
```

#69 Sqrt(x)

EASY

PROBLEM

Integer square root.

APPROACH

Binary search for largest i with i*i <= x.

COMPLEXITY

O(log x)

PYTHON SOLUTION

```
def mySqrt(x):
    lo, hi = 0, x
    while lo <= hi:
        mid = (lo + hi) // 2
        if mid * mid <= x < (mid + 1) * (mid + 1):
            return mid
        elif mid * mid > x: hi = mid - 1
        else: lo = mid + 1
```

#9 Palindrome Number

EASY

PROBLEM

Is int a palindrome? Negative is not. Don't convert to string (if asked).

APPROACH

Reverse half the digits. Compare with the remaining half.

COMPLEXITY

O(log n)

PYTHON SOLUTION

```
def isPalindrome(x):
    if x < 0 or (x % 10 == 0 and x != 0): return False
    rev = 0
    while x > rev:
        rev = rev * 10 + x % 10
        x //= 10
    return x == rev or x == rev // 10
```

#137 Single Number II

MEDIUM

PROBLEM

Every number appears 3 times except one. Find it.

APPROACH

Bitwise state machine: 'ones' and 'twos' track bits seen mod 3.

COMPLEXITY

O(n) time, O(1) space

PYTHON SOLUTION

```
def singleNumber(nums):
    ones = twos = 0
    for n in nums:
        ones = (ones ^ n) & ~twos
        twos = (twos ^ n) & ~ones
    return ones
```

#172 Factorial Trailing Zeroes

MEDIUM

PROBLEM

Number of trailing zeros in n!.

APPROACH

Count factors of 5 in n! (2s are more abundant). n/5 + n/25 + n/125 + ...

COMPLEXITY

O(log n)

PYTHON SOLUTION

```
def trailingZeroes(n):
    count = 0
    while n:
        n //= 5
        count += n
    return count
```

#201 Bitwise AND of Numbers Range

MEDIUM

PROBLEM

AND of all numbers from m to n.

APPROACH

Find common prefix of m and n's binary — that's the answer (all bits past the prefix vary).

COMPLEXITY

O(log n)

PYTHON SOLUTION

```
def rangeBitwiseAnd(m, n):
    shift = 0
    while m != n:
        m >>= 1
        n >>= 1
        shift += 1
    return m << shift
```

#50 Pow(x, n)

MEDIUM

PROBLEM

Implement x^n.

APPROACH

Fast exponentiation. Square and halve. Handle negative n by inverting.

COMPLEXITY

O(log n)

PYTHON SOLUTION

```
def myPow(x, n):
    if n < 0: x = 1/x; n = -n
    res = 1.0
    while n:
        if n & 1: res *= x
        x *= x
        n >>= 1
    return res
```

15. Cheatsheets & Templates

Print these. Tape them above your desk.

Pattern Selection Guide

Problem Description Contains...	Try This Pattern
sorted array, find pair / palindrome	Two Pointers
contiguous subarray/substring with property	Sliding Window
anything with 'distance' or 'window' k	Sliding Window or Hash Map
top k / kth largest / k closest	Heap (size k)
shortest path unweighted / level by level	BFS
all paths / count components / detect cycle	DFS or Union-Find
dependencies / build order / cycle detection	Topological Sort
all subsets / permutations / combinations / N-queens	Backtracking
min / max / count of ways / optimal substructure	Dynamic Programming
matrix with sorted rows/cols	Binary Search or two-pointer staircase
LRU/LFU/min-stack/median stream	Hash Map + Doubly Linked List, or two heaps
prefix / autocomplete / word lookup	Trie
next greater / smaller element	Monotonic Stack
range sum / range product queries	Prefix sum or Segment Tree

Complexity Quick Reference

Input n	Acceptable Complexity
n ≤ 10	O(n!) — permutations brute force
n ≤ 20	O(2^n) — subset enumeration
n ≤ 100	O(n^4)
n ≤ 500	O(n^3)
n ≤ 5,000	O(n^2)
n ≤ 10^6	O(n log n) or O(n)
n ≤ 10^8	O(n) — single pass
n > 10^8	O(log n) or O(1) — math/binary search

Python Built-ins for LeetCode

What You Need	Built-in
Sort	`sorted(it, key=fn, reverse=True), it.sort(...)`
Reverse iterable	`reversed(it), it[::-1]`
Counter / freq map	`collections.Counter(it)`

What You Need	Built-in
Default dict	`collections.defaultdict(list)`
Stack / queue / deque	`list (stack), collections.deque (both ends)`
Min/max heap	`heapq (min-heap; negate for max)`
Top k	`heapq.nlargest(k, it), nsmallest(k, it)`
Binary search	bisect.bisect_left, bisect.insort
Combinations / permutations	itertools.combinations, permutations
Cartesian product	`itertools.product(a, b)`
Accumulate (running sum)	`itertools.accumulate(it)`
Chain iterables	`itertools.chain(a, b, c)`
Pairs of adjacent	`zip(it, it[1:])`
Enumerate from i	`enumerate(it, start=i)`
Unicode point	`ord(c); chr(n)`
Divmod	`divmod(a, b)`
Math infinity	`float('inf'), float('-inf')`
Cache function results	functools.lru_cache or @cache

Common Code Patterns (Templates)

Pattern	Template (Python)
BFS	`q=deque([start]); seen={start}; while q: x=q.popleft(); for nei in adj(x): if nei not in seen: seen.add(nei); q.append(nei)`
DFS (recursive)	`def dfs(x): if base: return; for nei in adj(x): dfs(nei)`
Binary search leftmost	`lo,hi=0,n; while lo<hi: mid=(lo+hi)//2; if a[mid]<t: lo=mid+1; else: hi=mid; return lo`
Sliding window (variable)	`l=0; for r,x in enumerate(arr): update; while invalid: shrink l; track best`
Backtracking	`def bt(state): if done: record; return; for choice in choices: apply; bt(state); undo`
DP 1D	`dp=[0]*(n+1); for i in range(1,n+1): dp[i]=fn(dp[i-1],...); return dp[n]`
Topological sort (Kahn's)	`indeg=[0]*n; q=deque(i for i in range(n) if indeg[i]==0); while q: u=q.popleft(); for v in adj[u]: indeg[v]-=1; if indeg[v]==0: q.append(v)`
Union-Find	`parent[x]=x for all x; find(x): while parent[x]!=x: parent[x]=parent[parent[x]]; x=parent[x]; return x`
Monotonic stack (next greater)	`for i,x in enumerate(arr): while stack and arr[stack[-1]]<x: result[stack.pop()]=x; stack.append(i)`
Two heaps (median)	`lower=max-heap (negate); upper=min-heap; balance after each push`

Trick Notes / Common Pitfalls

Trap	Fix
Off-by-one in binary search	Use lo<hi (open) consistently or lo<=hi (closed). Pick ONE.
Integer overflow (other langs)	Python ints are unbounded — but watch when porting.
Mutating list while iterating	Iterate over a copy (list(d)) or build a new list.
Recursion depth in deep trees	Python default ~1000. Convert to iterative or sys.setrecursionlimit.
Using float('inf') for negative compare	Pair with float('-inf') correctly — don't mix up.
Forgetting to dedup in backtracking	Sort + skip nums[i]==nums[i-1] when not starting at i.
Using == when 'is' is faster (for sentinels)	node is None, item is SENTINEL.
Building strings with += in a loop	O(n²). Use ''.join(list).
dict.get(k) vs dict[k]	get() returns None; [] raises. Use [] when key must exist.
Forgetting to reset state in backtracking	Always undo the choice after recursion returns.
Missing edge cases: empty input, single elem	Always handle: [], [1], all same, sorted, reverse-sorted.